```
#Basic box_plot for one numerical variable showing distribution
ggplot(mtcars, aes(mpg))+geom_boxplot()
ggplot(mtcars, aes(mpg))+geom_boxplot(color="black", fill="blue")
ggplot(mtcars, aes(mpg))+geom_boxplot(color="blue")+coord_flip()

#Boxplot for Numerical value as per a categorical variable
ggplot(mtcars, aes(x=am, y=mpg))+geom_boxplot()

ggplot(mtcars, aes(x=mpg, y=factor(am)))+geom_boxplot()
ggplot(mtcars, aes(x=factor(am), y=mpg))+geom_boxplot()
ggplot(mtcars, aes(fill=factor(am), y=mpg))+geom_boxplot()
ggplot(mtcars, aes(x=factor(cyl), y=mpg ))+geom_boxplot()#less cyl
more average but consistency is less variation is more

#box color and border
ggplot(mtcars, aes(x=factor(cyl), y=mpg,
fill=factor(cyl)))+geom_boxplot()
ggplot(mtcars, aes(x=factor(cyl), y=mpg))+geom_boxplot(fill="red")
ggplot(mtcars, aes(x=factor(cyl), y=mpg,
color=factor(cyl)))+geom_boxplot()
#add exact points
ggplot(mtcars, aes(x=factor(cyl), y=mpg, fill=factor(cyl)))+
  geom_boxplot()+geom_point()
View(mtcars)
dim(mtcars)

#notched plot
ggplot(mtcars, aes(x=factor(cyl), y=mpg, fill=factor(cyl) ))+
  geom_boxplot(notch=TRUE)

#Box plot with multiple groups
ggplot(mtcars, aes(x=factor(cyl), y=mpg, fill=factor(cyl) ))+
  geom_boxplot()+facet_wrap(~am)

ggplot(mtcars, aes(x=factor(cyl), y=mpg, fill=factor(am)))+
  geom_boxplot()

ggplot(mtcars, aes(x=factor(cyl), y=mpg, fill=factor(am) ))+
  geom_boxplot()+facet_wrap(~vs)
```

```
# Basic Histogram

ggplot(mtcars, aes(mpg))+geom_histogram()
ggplot(mtcars, aes(mpg))+geom_histogram(color="black", fill="white")
ggplot(mtcars, aes(mpg))+geom_histogram(color="black", fill="white",
binwidth=5)
ggplot(mtcars, aes(fill=factor(am), mpg))+geom_histogram()
#explore different binwidth(decide value as per range)


ggplot(mtcars, aes(mpg))+geom_histogram(color="blue",
fill="white")+coord_flip()

#Histogram for Numerical value as per a categorical variable
## Change histogram plot line colors by groups
ggplot(diamonds, aes(price))+geom_histogram()
ggplot(diamonds, aes(price))+geom_boxplot()

ggplot(diamonds, aes(price, fill=cut ))+geom_histogram()
ggplot(diamonds, aes(price, fill=cut))+geom_boxplot()
#Adding mean line
ggplot(diamonds, aes(price, fill=cut
))+geom_histogram()+geom_vline(aes(xintercept=mean(price),
color="red"))

ggplot(diamonds, aes(price, color=cut ))+geom_histogram()# border
instead of fill

ggplot(diamonds, aes(price, fill=cut
))+geom_histogram()+facet_wrap(~clarity)
ggplot(diamonds, aes(price, fill=cut
))+geom_histogram()+facet_wrap(~color)

# Violin

ggplot(diamonds, aes( cut,price))+geom_violin()
ggplot(diamonds, aes( price,cut))+geom_point()+
  geom_violin(fill="red", color="white")
ggplot(diamonds, aes(
price,cut))+geom_point()+geom_violin(trim=FALSE)
ggplot(diamonds, aes( price,cut,fill=cut ))+geom_point()+
  geom_violin(trim=FALSE)+
  stat_summary(fun.y=median, geom="point", size=2, color="red")
```

```
ggplot(diamonds, aes( price,cut,color=cut ))+geom_point()+
  geom_violin(trim=FALSE)+
  stat_summary(fun.y=median, geom="point", size=2, color="red")


# violin plot with mean points
ggplot(diamonds, aes( price,cut))+geom_point()+
  geom_violin(trim=FALSE)+
  stat_summary(fun.y=mean, geom="point",  size=2)
# violin plot with median points
ggplot(diamonds, aes( price,cut))+geom_point()+
  geom_violin(trim=FALSE)+
  stat_summary(fun.y=median, geom="point", size=2, color="red")



#Pie Chart
group_by(diamonds, cut) %>% summarise(Count=n()) %>%
  ggplot(aes(x=" ", y=Count, fill=cut))+geom_bar(stat="identity")

group_by(diamonds, cut) %>% summarise(Count=n()) %>%
  ggplot(aes(x=" ", y=Count, fill=cut))+geom_bar(stat="identity")+
  coord_polar("y", start=0)

slice(diamonds, 1:500) %>% group_by(cut) %>% summarise(Count=n())
%>%
  ggplot(aes(x=" ", y=Count, fill=cut))+geom_bar( stat="identity")+
  coord_polar("y", start=0)+
  geom_text(aes(label = Count),position = position_stack(vjust =
0.5), color="white", size=4)

#A jitter plot is used when you have discrete or categorical
variables, and many points may overlap, leading to overplotting

ggplot(diamonds, aes(x= cut, y=price))+
  geom_point()
# understand difference between scatter and jitter plot
set.seed(123)
ggplot( diamonds, aes(x= cut, y=price))+
  geom_jitter()#many overlapping points can be observed

ggplot( diamonds, aes(x= cut, y=price))+geom_jitter(alpha=0.5)
```

```
ggplot( diamonds, aes(x= cut, y=price))+
  geom_jitter(alpha=0.1, height =0.3) #samples can be used

#consider less observations
filter(diamonds, price>15000) %>%
  ggplot(  aes(x= cut, y=price))+
  geom_jitter(alpha=0.2, height =0.4 )#observe overlapping

ggplot(mtcars, aes(cyl, mpg))+geom_jitter(height =0.2, width=0.5)
ggplot(mtcars, aes(factor(cyl), factor(am)))+geom_jitter(height
=0.2, width=0.5, alpha=0.5)
ggplot(mtcars, aes(cyl, am, color=factor(cyl)))+
  geom_jitter(height =0.2, width=0.5, alpha=0.5)

#grid form
P1=ggplot(mtcars,aes(mpg, hp) )+geom_point()
P1
P2=ggplot(mtcars,aes(mpg, wt) )+geom_point()
P2
P3=ggplot(mtcars,aes(mpg, disp) )+geom_point()
P3
P4=ggplot(mtcars,aes(mpg, drat) )+geom_point()
P4

dev.off()
library(grid)
viewport<- viewport(
  layout=grid.layout(2,2))
pushViewport(viewport)
print(x=P1,
      vp=viewport(
        layout.pos.row = 1,
        layout.pos.col=1))
print(x=P2,
      vp=viewport(
        layout.pos.row = 2,
        layout.pos.col=1))
print(x=P3,
      vp=viewport(
        layout.pos.row = 2,
        layout.pos.col=2))
print(x=P4,
```

```
vp=viewport(
  layout.pos.row = 1,
  layout.pos.col=2))
```